

Architecting a Blue Team Home Lab: A Floating-Phase Guide to Building SOC Skills with Wazuh

This report outlines a comprehensive, mentor-guided implementation plan for a home-based cybersecurity lab project. The objective is to build a robust, multi-device environment focused on blue team defense skills, specifically targeting roles such as Security Operations Center (SOC) Analyst, Network Analyst, or Network Security Analyst. The plan is structured over an 11-week period, accommodating a 15-hour weekly commitment within a maximum timeline of two months and fifteen days. It adheres to a "floating-phase" model, which allows for the strategic overlap of project stages once minimum viability gates are achieved. This approach mirrors real-world SOC workflows, where analysts do not wait for perfect conditions before beginning defensive operations. The project leverages the user's existing hardware—a 16GB RAM Pop!_OS dual-boot PC, a 4GB RAM Windows PC, and a ~12GB RAM Arch Linux laptop—and incorporates a future Raspberry Pi addition. The core technology stack is Wazuh, deployed as a Security Information and Event Management (SIEM) system, supplemented by Atomic Red Team for threat simulation. Every recommendation is grounded in objective logic, provides clear justification, and includes necessary technical commands to guide the user through their first major hands-on project.

Phase 0: Foundational Architecture and Environment Setup

The success of this entire project hinges on a meticulously planned and executed foundational architecture. This initial phase is not merely about turning on machines; it is about designing a stable, secure, and interconnected laboratory that serves as the bedrock for all subsequent activities. The goal of Phase 0, spanning one week with a commitment of approximately 12 hours, is to have all virtual machines running and the network properly configured to support seamless communication between the Wazuh server, various agent endpoints, and external threat intelligence sources. This phase addresses the critical decision of deploying the Wazuh server within a virtual machine

(VM) on the 16GB RAM desktop PC, transforming the 4GB RAM PC into a bare-metal Linux agent, and repurposing the laptop as a dual-role workstation hosting a Windows client VM and a secondary analysis tool. This allocation strategy optimizes the user's limited hardware resources to generate diverse log data from multiple operating systems, a crucial factor for building a compelling cybersecurity portfolio [8](#) [11](#).

The central decision in this phase is the deployment location for the Wazuh server. While the user initially expressed a preference for a native hypervisor, the recommended approach is to deploy the Wazuh server inside a VM on the 16GB RAM PC rather than on the host operating system. This choice is rooted in pedagogical principles and long-term stability. First, isolation is paramount. The VM acts as a sandbox, preventing any misconfiguration or instability within the Wazuh application from directly impacting the host Pop!_OS system, which the user will continue to use for other tasks. Second, VM snapshots provide an invaluable safety net. Before undertaking high-risk activities like testing new custom Wazuh rules—which can sometimes cause unexpected behavior—the entire state of the Wazuh server can be saved and quickly reverted if something goes wrong. This capability is non-existent in a bare-metal installation and is critical for a learning environment where experimentation is encouraged. Third, resource allocation is manageable; allocating 8GB of RAM to the Wazuh VM leaves a healthy 8GB for the host OS and other applications, ensuring the user's personal computing experience is not significantly impacted. Finally, a VM-based deployment enhances reproducibility, a key attribute for portfolio projects, as the entire environment can be documented and recreated elsewhere more easily. The performance overhead of virtualization for a SIEM workload, which is primarily I/O and indexing, is negligible on a system with 16GB of RAM, making the benefits of isolation and snapshotting far outweigh any minor performance cost.

The following table details the proposed hardware and software allocation, which forms the architectural blueprint for the project.

Device	Primary Role	Operating System	Secondary Role / Additional Function
16GB RAM PC	Wazuh Server Host	Pop!_OS (Host OS)	Casual web browsing is acceptable during light loads, but may slow down during intensive threat simulations.
4GB RAM PC	Bare-Metal Wazuh Agent	Pop!_OS (Installed via Dual Boot)	Provides a dedicated, low-overhead Linux endpoint for generating syslog and auditd data. Avoids the performance penalty of virtualization on this limited-resource machine.
12GB RAM Laptop	Windows Client VM Host & Secondary Analysis Workstation	Pop!_OS (Primary OS)	Runs a Windows 10/11 VM to simulate a corporate endpoint and also hosts a local Wazuh agent for direct log ingestion. Ensures stability with an LTS-based distribution.

Network configuration is another cornerstone of Phase 0. All three devices—the Wazuh server VM, the 4GB PC agent, and the laptop agent/client—must reside on the same local area network (LAN) to communicate effectively . As the administrator of the home router, you have the authority to configure port forwarding, which will be essential for certain aspects of the project . For this lab, the most straightforward approach is to assign static IP addresses to each physical and virtual device. This ensures predictable communication and simplifies firewall rule creation and agent configuration. You can achieve this either by configuring DHCP reservations on your router or by setting static IPs directly on each operating system. The Wazuh server, being the central hub, will need to be accessible by all agents. The default ports for Wazuh communication are TCP/1514 for agent-server communication and TCP/55000 for the active response functionality. These ports must be open on the Wazuh server's firewall. Additionally, when the Windows VM is hosted on the laptop, it will need access to the LAN to send its logs to the Wazuh server. Most hypervisors, including VirtualBox and VMware, use a NAT or Bridged networking mode by default. Bridged networking is often preferable for lab environments as it makes the VM appear as a separate device on the network, similar to a physical machine, which simplifies routing and firewall management.

The following table provides a summary of the key configuration steps for Phase 0.

Task	Description	Rationale
1. Install Hypervisor	Install VirtualBox or VMware Workstation Player on the 16GB RAM PC's Pop!_OS host OS.	This creates the virtualization layer needed to run the Wazuh server VM and the Windows client VM on the laptop.
2. Create Wazuh Server VM	Provision a new VM with 8 GB RAM, 2 CPU cores, and at least 40 GB of disk space. Install Pop!_OS on it.	Allocates sufficient resources for the Wazuh server to function smoothly without hindering the host OS.
3. Assign Static IPs	Configure static IP addresses for the Wazuh server VM, the 4GB RAM PC (after installing Pop!_OS), and the laptop on your home network.	Ensures consistent network addressing, which is critical for reliable communication between the SIEM and all agents.
4. Install Wazuh Server	Follow official Wazuh documentation to install the Wazuh server package on the newly created VM.	This establishes the central component of the security monitoring infrastructure.
5. Prepare Endpoint Machines	On the 4GB RAM PC, perform a clean installation of Pop!_OS. On the laptop, replace the current Arch Linux installation with Pop!_OS.	Using a stable, Long-Term Support (LTS) distribution like Pop!_OS (based on Ubuntu 22.04) mitigates concerns about rolling-release instability during the 2.5-month project duration.
6. Verify Network Connectivity	From each endpoint machine (physical and virtual), use <code>ping</code> to confirm connectivity to the Wazuh server's static IP address.	Basic network troubleshooting to ensure that agents can reach the server to register and send data.
7. Document the Architecture	Create a diagram or textual description of your final lab architecture, noting IP addresses, roles, and connections.	This documentation is the first deliverable for your portfolio and provides clarity for all subsequent phases.

By the end of this week, you will have a physically and logically separated lab environment. The Wazuh server will be running in its own VM, isolated from your daily

workflow. The other machines will be prepared to act as dedicated agents, providing a rich variety of data sources. This solid foundation will allow you to transition smoothly into Phase 1, where you will focus on populating this architecture with the Wazuh software itself. Remember, this initial investment in planning and setup prevents countless hours of frustration later on, especially when dealing with complex configurations and troubleshooting network-related issues.

Phase 1: Wazuh Server and Agent Deployment

With the foundational architecture in place, Phase 1 focuses on the core task of deploying and configuring the Wazuh software across the entire environment. Spanning two weeks (Weeks 2-3) with a commitment of approximately 20 hours, this phase aims to get a fully functional Wazuh server and agents communicating with each other. The successful completion of this phase means that log data from every designated endpoint—your Wazuh server VM, the 4GB RAM bare-metal PC, and the laptop running both a Windows VM and a Pop!_OS agent—is flowing into the Wazuh server and appearing in its dashboard. This marks the first major milestone and establishes the baseline operational capability of your home lab. The primary learning objectives here are gaining proficiency with the Wazuh stack (manager, indexer, dashboard), understanding agent-server registration, and mastering the initial configuration of the Wazuh agent on different platforms.

The first step is to complete the installation of the Wazuh server on the VM provisioned in Phase 0. The Wazuh stack consists of several key components, and for a home lab, the simplest way to deploy them is using the Wazuh installer, which sets up a combined manager and indexer instance by default. You will connect to your Pop!_OS VM and follow the official installation guide. This process typically involves downloading the installer script and running it with administrative privileges. During the installation, you will be prompted to set a password for the Wazuh API and Kibana, which are essential for managing the system and visualizing data. Once the server installation is complete, you must start the services. Use commands like `sudo systemctl status wazuh-manager` and `sudo systemctl status wazuh-indexer` to verify that both processes are active and running without errors. Any failures at this stage should be addressed immediately by checking the service logs located in `/var/ossec/logs/`.

Next, you will deploy and configure the Wazuh agents on all three endpoints. Each agent must be pointed to the static IP address of your Wazuh server to register successfully. The configuration process differs slightly between operating systems.

On the **Wazuh server VM itself**, you will install the agent package. This is useful for monitoring the health and security events of the SIEM server itself. After installing the agent, you edit its main configuration file, typically located at `/var/ossec/etc/ossec.conf`. Inside this file, you will find a `<client_configs>` section where you add the server's IP address:

```
<client_configs>
  <server>SERVER_STATIC_IP_ADDRESS</server>
</client_configs>
```

Replace `SERVER_STATIC_IP_ADDRESS` with the actual IP you assigned to your Wazuh server VM in Phase 0. After saving the file, restart the agent service with `sudo systemctl restart wazuh-agent`.

On the **4GB RAM PC** (now running Pop!_OS bare-metal), the process is identical. Install the Wazuh agent package for Linux, edit the `ossec.conf` file to point to the server's IP, and restart the service. This machine will now begin sending its system logs (`syslog`) and file integrity check data to your central SIEM.

The **laptop** has a dual role. First, you will install the Wazuh agent on its Pop!_OS OS in the same manner as the 4GB PC. This provides a third, distinct Linux data source. Second, on this same laptop, you will install a full version of Windows 10 or 11 inside a VirtualBox or VMware VM. Within this Windows VM, you will then install the Wazuh agent for Windows. The Windows agent installation is a simple executable that guides you through the setup. During the installation wizard, you will be asked for the server's IP address. Enter the static IP of your Wazuh server VM. The Windows agent is particularly powerful because it can monitor Event Logs, Registry changes, and file integrity, providing a wealth of data for a SOC analyst to analyze .

Once all agents are installed and pointing to the server, they must be registered. The Wazuh manager needs to explicitly accept each agent's connection request. You can manage this from the Wazuh server VM by using the `manage_agents` utility, which is a command-line tool for adding, listing, and deleting agents. Run `sudo /var/ossec/bin/manage_agents` and select option I to import an existing agent key. You will need to retrieve the agent key from each endpoint. On Linux agents, the key is located at `/var/ossec/etc/client.keys`. You can copy this file's contents and paste it when prompted. On Windows agents, the key is typically found in `C:\Program Files`

(x86)\ossec-agent\client.keys. After importing the keys, select option E to list the added agents. You should see all four of your agents (server, 4GB PC, laptop OS, and laptop's Windows VM) listed. Finally, select option Q to quit the utility. The manager will now initiate a connection to each agent, and you should start seeing data appear in the Wazuh dashboard.

The following table summarizes the key tasks and commands for Phase 1.

Task	Platform	Command / Action	Purpose
Install Wazuh Server	Wazuh Server VM (Pop!_OS)	Run the official Wazuh installer script. Set API/Kibana passwords. Start services with <code>systemctl</code> .	Installs the core SIEM components (manager, indexer).
Install Wazuh Agent	4GB RAM PC (Pop!_OS)	Install <code>wazuh-agent</code> package. Edit <code>/etc/ossec.conf</code> to add server IP. Restart agent.	Prepares the bare-metal Linux agent to send data.
Install Wazuh Agent	Laptop (Pop!_OS OS)	Install <code>wazuh-agent</code> package. Edit <code>/etc/ossec.conf</code> to add server IP. Restart agent.	Adds another distinct Linux data source to the SIEM.
Install Wazuh Agent	Laptop (Windows VM)	Run the Wazuh agent installer for Windows. Provide the server's static IP during setup.	Monitors Windows-specific events like Event Logs and Registry changes.
Register Agents	Wazuh Server VM	Use <code>/var/ossec/bin/manage_agents</code> . Import keys from each agent and list registered agents.	Authorizes agents to send data to the Wazuh manager.
Verify Communication	Wazuh Dashboard (Browser)	Navigate to 'Agents' tab. Confirm all agents show as 'Active'. Check 'Events' tab for incoming log data.	Confirms successful deployment and data flow.

A common issue during this phase is firewall blocking. Ensure that the firewall on the Wazuh server VM allows incoming traffic on port 1514 (and 55000 for active response). You can use `sudo ufw status` on the Pop!_OS server to check and modify rules if necessary. Another frequent problem is incorrect IP addresses in the agent configuration files. Double-check every `ossec.conf` file and every Windows agent setup prompt for typos. Patience and methodical verification are key. By the end of Week 3, you should have a fully populated Wazuh environment with five distinct data streams feeding into it. This achievement provides a strong sense of accomplishment and sets the stage for the more analytical work of log ingestion and threat simulation in the following phases.

Phase 2: Heterogeneous Log Ingestion and Data Normalization

Phase 2, scheduled for Weeks 3-4 with a commitment of around 15 hours, transitions from basic deployment to the critical task of enriching your SIEM with meaningful, actionable data. While Phase 1 ensured that raw log data was flowing into the Wazuh

server, Phase 2 is about making that data useful. The goal is to have all intended log sources—including Windows Event Logs, Linux system logs (`syslog`, `auth.log`), and potentially application-specific logs—properly ingested, parsed, and normalized so they can be correlated effectively. A minimum viable completion gate for this phase would be having at least Windows and Linux endpoint logs visible and searchable in the Wazuh dashboard, forming a baseline dataset for analysis. This phase is fundamental because the quality of your detections and investigations in later stages will be directly proportional to the quality and comprehensiveness of your data pipeline.

For the **Windows endpoints** (the Windows VM on your laptop), the Wazuh agent comes with built-in capabilities to monitor Windows Event Logs. However, to capture a wide range of security-relevant events, you must configure the agent to actively listen for specific event IDs. This is done by editing the `ossec.conf` file on the Windows agent. You will add `<localfile>` sections to define the paths to the logs you want to monitor and `<rule>` groups to specify the event IDs of interest. For a SOC analyst, key areas to focus on include account logon/logoff events (Event ID 4624, 4625), account management (Event ID 4720, 4726), object access (e.g., 4663 for file/folder access), and policy changes (Event ID 4719). An example configuration snippet might look like this:

```
<localfile>
  <location>Security</location>
  <log_format>eventchannel</log_format>
  <tags>win_security</tags>
</localfile>

<ruleset>
  <group name="win_security">
    <rule id="1001" level="10">
      <if_sid>11001</if_sid>
      <match>EventID="4624"</match>
      <description>New successful logon detected.</description>
    </rule>
    <rule id="1002" level="12">
      <if_sid>11001</if_sid>
      <match>EventID="4625"</match>
      <description>Failed login attempt detected.</description>
    </rule>
  </group>
</ruleset>
```

After making these changes, restart the Wazuh service on the Windows VM. You should begin to see alerts with levels 10 and 12 appearing in the Wazuh dashboard, corresponding to successful and failed logons. Experiment by logging into the Windows VM to generate some 4624 events and try incorrect passwords to generate 4625 events. This confirms your configuration is working.

For the **Linux endpoints** (the 4GB RAM PC and the laptop's Pop!_OS OS), the Wazuh agent monitors standard system logs by default. The primary sources are `/var/log/syslog` and `/var/log/auth.log`. The agent uses decoders to parse these logs into structured fields (e.g., timestamp, hostname, program name, message content). For enhanced security monitoring, you should enable and configure additional modules. One of the most important is the `auditd` decoder, which parses output from the Linux Audit Framework. To enable it, you can either use the pre-configured rules in Wazuh or manually add the decoder to `ossec.conf`. Furthermore, you can use the Wazuh agent to perform File Integrity Monitoring (FIM) on critical system files and directories, such as `/etc/passwd`, `/etc/shadow`, and binaries in `/bin` and `/usr/bin`. This is configured in the `<fim>` section of `ossec.conf`. FIM alerts will notify you of any unauthorized modifications to these files, a classic indicator of a compromise.

The following table outlines key log sources to target and their typical locations or methods of collection.

Log Source	Platform	Configuration Method	Key Events / Indicators
Windows Security Log	Windows VM	<code><localfile></code> in <code>ossec.conf</code>	Logons (4624, 4625), Account Creation (4720), Policy Changes (4719) 4
Linux Syslog / Auth.log	Pop!_OS Endpoints	Enabled by default in Wazuh agent	SSH logins, sudo usage, system service status
Linux Audit Framework (auditd)	Pop!_OS Endpoints	Enable <code>auditd</code> decoder in Wazuh <code>ossec.conf</code>	System calls, file access, permission changes
File Integrity Monitoring (FIM)	Pop!_OS Endpoints	Configure <code><fim></code> section in Wazuh <code>ossec.conf</code>	Unauthorized changes to critical system files
Application Logs	Pop!_OS Endpoints	Add <code><localfile></code> entries for app-specific logs	e.g., Apache/Nginx access/error logs, database logs

As you configure these data sources, you will likely encounter situations where logs are not appearing as expected. The Wazuh debugger is an invaluable tool for this. It can be accessed via the Wazuh server's command line and allows you to simulate how the agent will process a given log file, showing you exactly which decoder and rule it matches. This helps pinpoint configuration errors in `ossec.conf`. Another useful technique is to use the Wazuh API to query the raw JSON data for recent events and inspect the parsed fields. This gives you direct insight into what the system sees. For example, you can fetch

events for a specific agent ID and examine the `data` object to see if fields like `win_eventid` or `log` are populated correctly. Mastering these diagnostic tools is a key skill for any analyst.

By the end of Phase 2, your SIEM will be far more than just a data sink; it will be an observability platform capable of generating structured, security-relevant alerts from a diverse set of endpoints. This enriched dataset is the raw material for the next phase, where you will begin to test its effectiveness by simulating real-world attacks.

Phase 3: Threat Simulation and Observational Analysis

Phase 3, spanning two weeks (Weeks 5-6) with a commitment of approximately 25 hours, marks the shift from passive data collection to active, adversarial testing. The objective is to use Atomic Red Team, a library of small, standalone tests of adversarial tactics and techniques, to simulate a variety of cyberattacks against your lab environment. This phase embodies the iterative cycle of modern SOC work: simulate an attack, observe the resulting activity in your logs, and then engineer a detection for it. Successfully completing this phase means you have tested over ten distinct attack techniques and begun the process of observing their signatures within your Wazuh SIEM. This practical experience is invaluable, as it teaches you to think like an attacker while simultaneously strengthening your defensive posture.

Before beginning, you must prepare your environment. On your Wazuh server VM, ensure that the Wazuh Active Response feature is enabled. Active Response allows the Wazuh manager to automatically execute scripts in response to specific security alerts, which will be useful for containment actions during your simulations. Next, clone a snapshot of your Wazuh server VM. This is a critical safety measure. If a simulated attack causes unintended side effects or if you make a mistake in your Wazuh configuration, you can instantly revert to a clean state without having to rebuild your entire environment from scratch. This practice reinforces the importance of a controlled and recoverable lab.

To use Atomic Red Team, you will need to install it on a control machine. Your laptop, with its Pop!_OS installation, is an ideal candidate. Clone the Atomic Red Team repository from GitHub (`git clone https://github.com/redcanaryco/atomic-red-team.git`) and familiarize yourself with its directory structure. The framework is organized by ATT&CK technique (e.g., T1059.001 for PowerShell command execution). Each subdirectory contains a test YAML file that defines the adversary action

to be performed. You can run these tests using a runner script provided by Atomic Red Team.

Your strategy should be methodical. Begin with less invasive techniques and gradually move to more disruptive ones. Here is a suggested sequence of ten techniques to test, along with the expected logs to watch for in your Wazuh dashboard:

1. T1059.001 - PowerShell Command:

- **Action:** Execute a simple command like ``Get-Process`` via PowerShell.
- **Observation:** Look for alerts in the Wazuh Active Responses tab related to PowerShell command execution. Check the Windows Security log for Event ID 4104, which records script block logging.

1. T1055 - Process Injection:

- **Action:** Use a tool like ``ReflectivePEInjection`` (a known good tool for testing) to inject code into a benign process like ``notepad.exe``.
- **Observation:** Monitor for alerts from the FIM module on your Windows VM regarding the creation of child processes. Look for suspicious parent-child process relationships in the Wazuh Active Responses tab.

1. T1071.001 - Application Layer Protocol: Web Protocols:

- **Action:** Use ``curl`` or ``Invoke-WebRequest`` in PowerShell to download a harmless file from a public URL.
- **Observation:** Watch for outbound connection alerts from your Windows VM. The Wazuh agent can detect and alert on DNS requests and network connections made by applications.

1. T1548.002 - Abandonware:

- **Action:** Create a simple batch file containing a benign command and save it to a USB drive or shared network location.
- **Observation:** Monitor for alerts related to autorun.inf files or alerts triggered by the execution of scripts from unusual directories.

1. T1003 - OS Credential Dumping: LSASS Memory:

- **Action:** Use a tool like ``procdump`` to dump the memory of the Local Security Authority Subsystem Service (LSASS) process. This is a high-privilege action.

- **Observation:** This will likely trigger multiple alerts. The FIM module may alert on the creation of the dump file. Antivirus/EDR-like features in Wazuh may flag the ``procdump`` process itself. Analyze the correlation of these alerts.

1. T1134 - Access Token Manipulation:

- **Action:** Use a tool like ``runas`` or a more advanced token manipulation utility to impersonate another user.
- **Observation:** Look for anomalies in logon session data (Event ID 4624) and potential privilege escalation alerts.

1. T1027 - Obfuscated Files or Information: Base64:

- **Action:** Encode a simple text string into Base64 and decode it using PowerShell (`[System.Convert]::FromBase64String(...)``).
- **Observation:** This may not generate a default Wazuh alert, but it is a prime candidate for the next phase: detection engineering.

1. T1566.002 - Credentials in Password Stores:

- **Action:** Attempt to access credentials stored in the Windows Credential Manager.
- **Observation:** Monitor for specific API calls or registry accesses associated with credential retrieval.

1. T1546.008 - Screen Capture:

- **Action:** Run a screen capture utility.
- **Observation:** Look for alerts related to the execution of screenshot tools and their associated network activity if they attempt to exfiltrate data.

1. T1074 - Hidden Files and Directories:

- **Action:** Create a hidden file on a local drive.
- **Observation:** This may require custom FIM rules to detect, as it doesn't always generate a standard system event.

During and after each test, dedicate significant time to observational analysis. Open the Wazuh dashboard and use the search bar to filter for alerts related to the technique you just executed. Examine the full JSON event for each alert to understand the context. Ask yourself: "What specific piece of information in this log led to the alert?" "Could this alert be generated by legitimate administrative activity?" "How could I tune this rule to reduce false positives?" This analytical mindset is what separates a curious hobbyist from a proficient SOC analyst. Keep a detailed log of your experiments, noting the technique

tested, the commands used, and the corresponding Wazuh alerts generated. This documentation will form the basis for your custom rules and investigation playbooks in the following phases.

Phase 4: Custom Detection Rule Engineering

Following the active threat simulation of Phase 3, Phase 4, scheduled for Weeks 7-8 with a commitment of approximately 30 hours, focuses on the core defensive skill of detection engineering. The objective is to move beyond Wazuh's default rules and create your own custom rules to detect malicious activity that may not have been caught, or to improve the accuracy of existing detections. Successfully completing this phase means you have created and validated over twelve custom Wazuh rules. This process transforms your SIEM from a generic monitoring tool into a bespoke defense system tailored to the specific threats you've identified. It is a deeply analytical exercise that requires you to synthesize your observations from Phase 3 into precise, actionable logic.

The foundation of any custom rule is a clear understanding of the adversary's behavior and its unique signature in the data. When you ran the Base64 obfuscation test (T1074.002) in Phase 3, you likely did not receive a default alert. This is a perfect opportunity for custom rule creation. The signature for this behavior might be the execution of a PowerShell command that contains a high ratio of Base64 characters or the use of specific cmdlets known for encoding/decoding, such as `[System.Convert]::ToBase64String` or `[System.Convert]::FromBase64String`. Your task is to translate this behavioral signature into a Wazuh rule.

Wazuh rules are written in XML and are typically placed in the `local_rules.xml` file, usually located in `/var/ossec/etc/local_rules.xml` on the manager. This file is designed for custom additions and won't be overwritten during Wazuh updates. Let's walk through creating a rule for the Base64 obfuscation scenario.

First, you need to identify the data source. In this case, it's the `syscheck` or `active-response` data from the Windows agent, but more likely, it's the raw command line that was executed. Wazuh captures this in the `command` field of an event. Your rule will need to match on this field.

Here is an example of a custom rule you could add to `local_rules.xml`:

```
<!-- Rule 100001: Detect PowerShell Base64 String Conversion -->
<rule id="100001" level="12">
  <if_sid>55021</if_sid> <!-- Matches on a generic command execution alert
  <field name="agent.id" value="WINDOWS_AGENT_ID"/> <!-- Replace with your
  <field name="cmd" regex="true">powershell.*(\[System\.Convert\]::ToBase64
  <description><![CDATA[
    [PowerShell Base64 Obfuscation] A PowerShell command attempting to con
    This technique is often used to obfuscate malicious payloads.
    Source: User observation during Atomic Red Team testing.
  ]]></description>
</rule>
```

Let's break down this rule:

- ``: Assigns a unique ID and a severity level of 12 (high).
- `55021`: This links the rule to an existing Wazuh rule (SID 55021, "Command executed") to ensure it only applies to command execution events.
- ``: This restricts the rule to apply only to your Windows VM, avoiding noise from other agents.
- `...`: This is the core logic. It looks at the `cmd` field of the event and applies a regular expression to find the tell-tale .NET methods for Base64 conversion. The `regex="true"` attribute enables pattern matching.
- ``: A human-readable explanation of the alert, including why it's important and where the knowledge came from.

After saving the file, you must reload the rules on the Wazuh manager with `sudo systemctl reload wazuh-manager`. Then, re-run your Base64 obfuscation test. If configured correctly, you should now see a new, high-level alert in the Wazuh dashboard with your descriptive message.

You should repeat this process for at least eleven other techniques you observed in Phase 3. For example:

- **Rule for Scheduled Task Creation (T1053):** Create a rule that triggers on the execution of `schtasks.exe` with specific parameters.
- **Rule for Unusual Network Connection (T1048):** Create a rule that flags outbound connections to a non-standard port from a specific application.
- **Rule for Registry Modification (T1548.003):** Create a FIM rule that watches for writes to the `Run` keys in the registry.

The following table provides examples of custom rules you can create based on your observations.

Observed Behavior (from Phase 3)	Potential Signature	Suggested Rule Type	Example Trigger Condition
Base64 String Conversion	PowerShell command using [System.Convert]::ToBase64String	Regex Match on Command Line	<field name="cmd" regex="true">powershell.*\[System\.Convert\]::ToBase64String</field>
Suspicious Child Process	svchost.exe spawning powershell.exe	Correlation / Process Tree Alert	(Requires more advanced correlation rules)
Credential File Access	Access to .aws/credentials or .git-credentials	File Integrity Monitoring (FIM)	Watch for read/write access to these specific files.
DNS Tunneling Attempt	Unusually long DNS queries	Network Protocol Analysis (NPA) / Alert Correlation	Flag DNS queries where the question name length exceeds a threshold (e.g., 100 characters).

Testing and validation are crucial. Never assume a rule works as intended. Test it against both malicious and benign scenarios. For example, if you create a rule for `schtasks.exe`, test it by creating a legitimate scheduled task using the GUI and the command line to ensure it doesn't fire on legitimate use. If it does, you will need to refine your rule's conditions, perhaps by looking for specific, more malicious-looking arguments. This iterative refinement process is a hallmark of a skilled detection engineer. By the end of Phase 4, you will have a growing library of custom rules that demonstrate your ability to analyze threat data and translate it into effective defenses.

Phases 5-7: Playbook Development, Final Integration, and Portfolio Documentation

The final stretch of the project, encompassing Weeks 9, 10, 11, and ongoing, synthesizes all the preceding work into tangible, professional-grade artifacts. This concluding segment is divided into three interconnected phases: developing investigation playbooks (Week 9), performing final lab integration and cleanup (Week 10), and creating portfolio-ready documentation on GitHub (Week 11). The total commitment for these final weeks is approximately 68 hours, reflecting the depth and importance of producing

a polished, job-ready portfolio. The ultimate goal is to transform your hands-on lab experience into a compelling narrative of your skills and accomplishments, ready to be showcased to potential employers [7](#) [11](#) .

Phase 5: Investigation Playbook Development (Week 9, ~20 hours) An investigation playbook is a structured guide that walks a SOC analyst through the process of responding to a specific type of security incident [28](#) . It maps adversary tactics (from frameworks like MITRE ATT&CK) to the corresponding investigative steps, tools, and indicators of compromise (IoCs) that the blue team should collect. The objective of this phase is to author five detailed investigation documents, each centered around a different attack scenario you have simulated and detected .

A well-structured playbook typically follows a template that includes the following sections: 1. **Playbook Title & Summary:** A concise title (e.g., "Investigating PowerShell-based Lateral Movement") and a brief summary of the threat. 2. **ATT&CK Mapping:** A mapping of the playbook to specific MITRE ATT&CK techniques it covers (e.g., T1059 for Command and Scripting Interpreter, T1021.001 for Remote Services: SMB/Windows Admin Shares). 3. **Initial Detection:** Describes the specific Wazuh alert(s) that would trigger the playbook. Include the alert message, rule ID, and sample JSON data from the Wazuh dashboard. 4. **Scope of Investigation:** Defines the initial questions to answer (e.g., "Which host was the initial point of compromise?", "What user was active on the host?", "Were there any suspicious logon attempts?"). 5. **Evidence Collection & Analysis Steps:** This is the core of the playbook. It lists the exact commands or queries to run in your tools (Wazuh, PowerShell, etc.) to gather evidence. For example:

- Query Wazuh for all events from the affected agent ID in the last 24 hours.
- Search for related alerts, especially those with higher levels or from different agents.
- Use PowerShell on the endpoint to check running processes (``Get-Process``) and recent logon sessions (``query user``).
- Check the Windows Event Log for specific IDs related to the attack path.

1. **Containment & Eradication Actions:** Outlines the steps to stop the adversary's activity and remove them from the network. This could include disabling a user account, killing a malicious process, or blocking an IoC at the firewall.
2. **Lessons Learned:** A reflective section summarizing what was discovered, what went well, and what could be improved in the detection or response process.

Phase 6: Final Integration and Cleanup (Week 10, ~18 hours) This phase involves integrating the final piece of hardware and cleaning up the lab for documentation

purposes. The primary task is to integrate the Raspberry Pi, which you will receive in two weeks. The Raspberry Pi, running a Linux-based OS like Raspberry Pi OS (which is Debian-based), will serve as a lightweight IoT sensor in your environment [15](#). You will install the Wazuh agent on it, just as you did on the other Linux machines. This adds another distinct data source, demonstrating your ability to secure heterogeneous and potentially constrained devices.

Simultaneously, this week is dedicated to tidying up your lab. Go through your Wazuh manager and disable any rules or configurations that were used solely for testing and are no longer relevant to a production-like environment. This might include overly aggressive FIM rules or experimental Active Response scripts. Take a final snapshot of your Wazuh server VM to preserve the "gold master" configuration of your lab. This snapshot is a valuable asset for future reference or for recreating the lab if needed.

Phase 7: Portfolio Documentation (Week 11, ~10 hours + Ongoing) The final and most critical phase is packaging everything into a professional GitHub repository. This repository is your project's public face and must be meticulously organized. The objective is to have a GitHub repository ready for employers. Your repository should contain the following:

- **A Comprehensive README.md:** This is the entry point to your project. It should include a project title, a compelling overview of the lab's purpose and scope, a list of the skills demonstrated, and screenshots of your Wazuh dashboard showing key alerts and visualizations [7](#) [10](#). Link to your LinkedIn profile if applicable.
- **The Investigation Playbooks:** Store each of your five investigation playbooks as a Markdown (.md) file in a dedicated `/playbooks` directory. This makes them easy to read and navigate.
- **Custom Rules:** Place your custom Wazuh rules from `local_rules.xml` in a `/rules` directory. Include comments explaining the purpose of each rule.
- **Configuration Files:** Include sanitized versions of your key configuration files (e.g., `ossec.conf` snippets) in a `/configs` directory to showcase your setup.
- **The Lab Architecture Diagram:** Include a clear diagram (created in a tool like draw.io or Lucidchart) that visually represents your entire lab setup, as planned in Phase 0.
- **A Blog-style "Journal":** Consider adding a `/journal` or `/docs` folder with Markdown files that chronicle your journey through each phase. Documenting challenges you faced and how you overcame them adds a personal touch and demonstrates problem-solving skills.

By diligently working through these final phases, you will have not only completed a technically impressive project but also transformed it into a powerful career-building asset. Your home lab will stand as undeniable proof of your initiative, technical competence, and analytical mindset, perfectly aligning with the goals of becoming a skilled SOC analyst.

Reference

1. 7 GitHub projects for SOC Analysts to build cybersecurity skills https://www.linkedin.com/posts/gmfaruk_cybersecurity-socanalyst-infosec-activity-7356463949585764352-uYWB
2. 37 reader reviews of Bazzite... - DistroWatch.com <https://distrowatch.com/dwres.php?resource=ratings&distro=bazzite>
3. Changelog - The Linux Kernel Archives <https://www.kernel.org/pub/linux/kernel/v5.x/ChangeLog-5.15.190>
4. Analyzing Wazuh's Effectiveness in Server Monitoring - ResearchGate https://www.researchgate.net/publication/383329985_Real-time_Defense_Against_Cyber_Threats_Analyzing_Wazuh's_Effectiveness_in_Server_Monitoring
5. How To Build A SIEM with Suricata and Elastic Stack on Ubuntu 20.04 <https://www.digitalocean.com/community/tutorials/how-to-build-a-siem-with-suricata-and-elastic-stack-on-ubuntu-20-04>
6. Autonomous Federated Learning for Distributed Intrusion Detection ... <https://ieeexplore.ieee.org/iel7/6287639/10005208/10296918.pdf>
7. How to build a cybersecurity portfolio on GitHub - LinkedIn https://www.linkedin.com/posts/sania-khan-95754b20a_no-experience-build-a-cybersecurity-github-activity-7336350580807819264-gdzK
8. How to Create a Cybersecurity Portfolio - LinkedIn <https://www.linkedin.com/top-content/career/building-a-personal-portfolio/how-to-create-a-cybersecurity-portfolio/>
9. 791 reader reviews of MX Linux... - Distrowatch <https://distrowatch.com/dwres.php?resource=ratings&distro=mx>
10. How to avoid GitHub mess: 4 best practices for data pros - LinkedIn https://www.linkedin.com/posts/anita-makori_github-best-practices-for-data-analysts-activity-7376529648983363584-dZj3

11. Highlighting Personal Projects for Cybersecurity Job Applications <https://www.linkedin.com/top-content/career/building-a-personal-portfolio/highlighting-personal-projects-for-cybersecurity-job-applications/>
12. Mastering ROS 2 For Robotics Programming Design, Build ... - Scribd <https://www.scribd.com/document/912727269/Mastering-ROS-2-for-Robotics-Programming-Design-Build-Simulate-And-Prototype-Complex-Robots-Using-the-Robot-Operating-Lentin-Joseph-Jonathan-Cac>
13. linux - Debian Package Tracker <https://tracker.debian.org/linux>
14. DistroWatch Weekly <https://distrowatch.com/weekly.php?issue=20250623>
15. Model-driven Self-adaptive Deployment of Internet of Things ... <https://dl.acm.org/doi/full/10.1145/3549553>
16. 7 GitHub projects for SOC Analysts to improve cybersecurity skills ... https://www.linkedin.com/posts/waleed-ahmad-99b510238_linked-in-activity-7380618408083734528-BI3Y
17. SOC Home Lab: Hands-on Attack Detection & Defense - LinkedIn https://www.linkedin.com/posts/ahinsunil-m_soc-cyber-security-activity-7300442010962784256-SHQ1
18. Building a realistic Azure-based SOC lab | Bruno M. posted on the ... https://www.linkedin.com/posts/bruno-moulheres_cybersecurity-microsoftsentinel-azure-activity-7340454725089198081-G2uT
19. Building a Cybersecurity Home Lab — Part 4.2: Wazuh Installation ... <https://cybersecuritywriteups.com/building-a-cybersecurity-home-lab-part-4-2-wazuh-installation-configuration-785c67231e2b>
20. Reader Reviews - Distrowatch <https://distrowatch.com/dwres.php?resource=ratings&distro=opensuse>
21. Matter certification - Technical Documentation - Nordic Semiconductor https://docs.nordicsemi.com/bundle/ncs-2.9.1/page/nrf/protocols/matter/end_product/certification.html
22. [PDF] Release Notes for Oracle Linux 8.10 https://docs.oracle.com/cd/F22978_01/relnotes8.10/OL8-RELNOTES8-10.pdf
23. What ISSO job descriptions don't tell you - LinkedIn https://www.linkedin.com/posts/christopher-okpala-413565158_grc-cybersecuritycareers-ittocyber-activity-7335349528033734656-l7RU
24. DistroWatch Weekly <https://distrowatch.com/weekly.php?issue=20240603&mode=68>
25. Full Circle I220 08.2025 | PDF | Linux | Kernel (Operating System) <https://www.scribd.com/document/909555808/Full-Circle-I220-08-2025>

26. WARNING: The requested image's platform (linux/amd64) does not ... <https://stackoverflow.com/questions/72152446/warning-the-requested-images-platform-linux-amd64-does-not-match-the-detecte>
27. OSCP-style Red Team lab: Enumeration, Pivoting, Detection ... https://www.linkedin.com/posts/bassey-solomon-henry_cybersecurity-ocsp-soc-activity-7362697757356179456-m0QN
28. Integrating Wazuh and TheHive for SOC lab | Neema Benson ... https://www.linkedin.com/posts/neema-benson-kisanga-9405241aa_soclab-cybersecurity-wazuh-activity-7335164189558165504-9C0i
29. Solved a Real-World Cyberattack Simulation | Harsh Waghela ... https://www.linkedin.com/posts/harsh-max_lockbitransomware-cybersecurity-incidentresponse-activity-7301535333496557568-MFvI